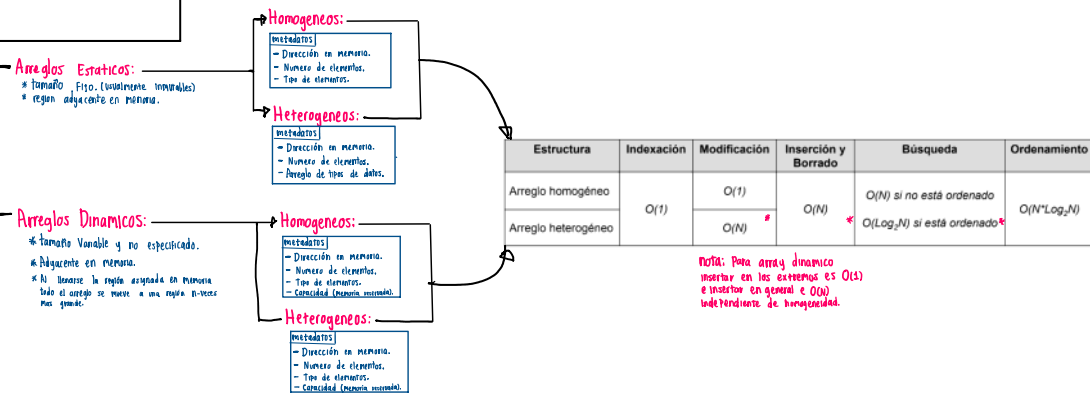
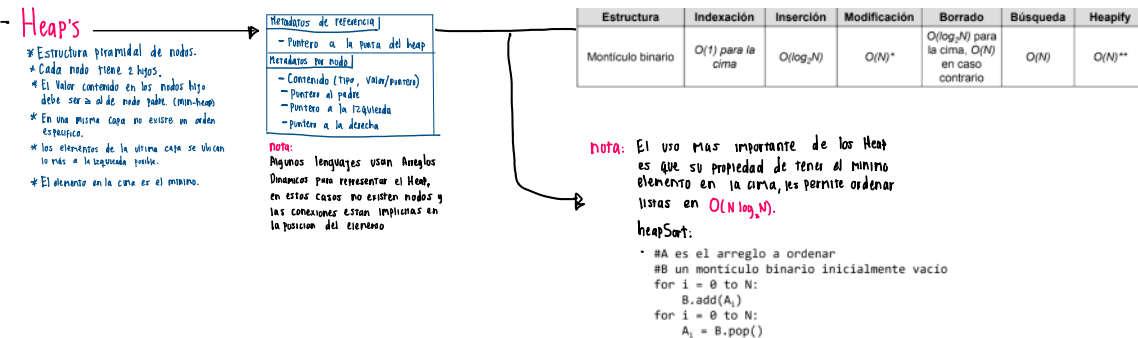
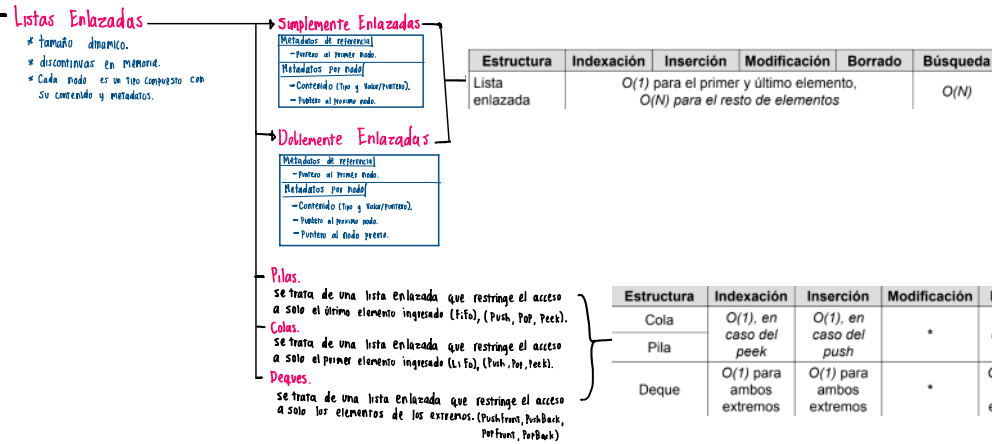




nota: se puede mejorar la eficiencia de búsqueda si el arreglo está ordenado, usando búsqueda binaria: $O(\log_2 N)$

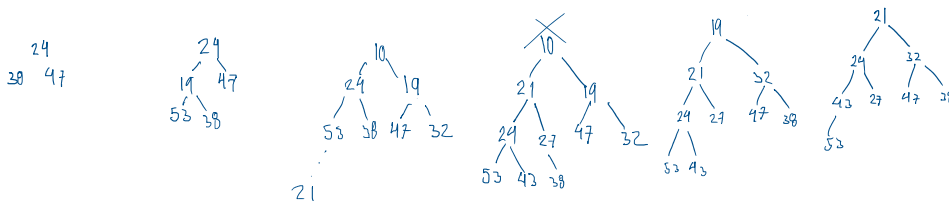
selectSort, bubbleSort, insertSort son "naive approaches" con eficiencias de $O(N^2)$.

mergeSort usa la filosofía de dividir y vencer, es recursivo y costoso en memoria, eficiencia: $O(N \log_2 N)$



1. (30%) Si se insertan uno a uno y en este orden estricto los siguientes elementos en un montículo binario ascendente (la cima es el menor) y luego se borra 2 veces la cima, ¿Cómo queda el montículo (dibújelo)?

38 24 47 53 19 32 10 21 43 27



2. Realice una prueba de escritorio al siguiente código y determine:

```

from bisect import bisect_left
import heapq

a = []
b = []
N = int(input())

for i in range(N):
    x = int(input())
    a.append(x)
    heapq.heappush(b, x)

a.sort()
for i in range(N):
    
```

A. (15%) Qué imprimiría si la entrada fuera:

7
23
35
14
35
40
23
35

0
1
2
3
4
5
6

B. (15%) Para un valor de N cualquiera, cuál es su eficiencia en notación Big Oh:

2. Realice una prueba de escritorio al siguiente código y determine:

```
from bisect import bisect_left
import heapq

a = []
b = []
N = int(input())

for i in range(N):
    x = int(input())
    a.append(x)
    heapq.heappush(b, x)

a.sort()
for j in range(N):
    z = heapq.heappop(b)
    print(bisect_left(a, z))
```

A. (15%) Qué imprimiría si la entrada fuera:

7
23
35
14
35
40
23
35

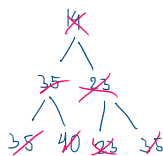
0
1
1
1
1
1
1
1

B. (15%) Para un valor de N cualquiera, cuál es su eficiencia en notación Big Oh:

$O(N \log N)$

1.5/1.5

b:



a: 14, 23, 23, 35, 23, 35, 40

0
1
1
1
1
1
1



1. Realice una prueba de escritorio al siguiente código y determine:

```
import heapq #monticulo binario
x = tuple(map(int, input().split()))
N = len(x)
y = []
for i in range(N):
    heapq.heappush(y, x[i])

while len(y) > 0:
    z = heapq.heappop(y)
    for j in range(N):
        if x[j] == z:
            print(j)
```

A. (15%) Qué imprimiría si la entrada fuera:

12 26 8 44 35 17
2
0
5
1
4

B. (15%) Para una entrada correspondiente a una secuencia de números cualesquiera (N en total), cuál es su eficiencia en notación Big Oh:

$O(N^2)$

17, 26, 8, 44, 35, 17

8, 12, 17, 26, 35, 44

Tiabeanie Mariabeanie de la Rochambeau Grunkwitz, también conocida como Princesa Bean es el personaje principal de la serie animada Desencanto, creada por Matt Groening, famoso por Los Simpsons. En la cuarta temporada, Bean pasa a compartir la corona con su padre, el Rey Zeg y decide ofrecer audiencias con los ciudadanos de Dreamland para escuchar sus peticiones. Con lo que no contaba es que todos querían ser escuchados y, peor aún, tenían más de una petición. Para no tardar tanto con cada quien, decidió que solo escucharía una petición por turno, los cuales durarían 5 minutos únicamente. Si alguien tenía más de una petición, tendría que hacer la fila de nuevo.

Si conocemos la cantidad de peticiones de cada ciudadano ¿le ayudarías a calcular cuánto tiempo le tomará para atender todas las peticiones? Supongamos que todas las audiencias duran exactamente los 5 minutos, que una ocurre inmediatamente después de la otra, y que Bean, pobrellita, no toma ningún descanso.

Entrada
La entrada comienza con una línea que la cantidad N de ciudadanos en la fila, no más de 10000. Luego sigue una línea que contiene N valores separados entre sí por un espacio en blanco y que corresponden a la cantidad de peticiones de cada ciudadano, por lo menos una y no más de 50. El primer valor corresponde a la cantidad de peticiones del ciudadano que está de primero en la fila, el segundo a la cantidad del que está de segundo y así sucesivamente.

Salida
La salida debe tener una única línea con el tiempo en minutos que tardará Bean en atender todas las audiencias.

Ejemplo de entrada

4
2 3 1 4

Ejemplo de salida

50

2 3 1 4
3 1 4 1
1 4 1 2
4 1 2
1 2 3
2 3
3 1
1 2
2
1
0

import deque

n = int(input())

q = deque(map(int, input().split()))

ctr = 0

while len(q) > 0:

a = n.popleft()

if a != 0:

ctr += 1

q.append(a-1)

Print(ctr*5)

from collections import deque

i = int(input())

q = deque(map(int, input().split()))

ctr = 0

while len(q) > 0:

a = q.popleft()

if a != 0:

ctr += 1

q.append(a-1)

print(ctr*5)

2. Realice una prueba de escritorio al siguiente programa:

```
N = int(input())
#A es un arreglo de tamaño N
A = tuple(map(int, input().split()))
i = 0
j = 2
x = 0
while i < 4:
    k = i
    while k < N:
        x += A[k]
        k += j
    j += 1
    i += 1
    print(x)
```

$O(2)$

$O(N)$

a): $O(N)$

b): output:

20
38
50
55

Luego determine: a. (10%)Cuál es la eficiencia en notación Big Oh en términos de N; y b. (20%) Qué imprimiría (déjelo explícito) si la entrada fuera la siguiente:

8
2 3 4 5 6 7 8 9

3. Realice una prueba de escritorio al siguiente programa:

```
from collections import deque
```

a: $O(N)$

3. Realice una prueba de escritorio al siguiente programa:

```
from collections import deque
N = int(input())
#X es una pila de tamaño N
X = deque(map(int, input().split()))
Y = deque()
i = 1
while len(X) > 0:
    e = X.pop()
    if i % 3 != 0:
        Y.append(e) #push
    i += 1
j = 1
while len(Y) > 0:
    e = Y.pop()
    if j % 2 != 0:
        print(e)
    j += 1
```

Luego determine: a. (10%) Cuál es la eficiencia en notación Big Oh en términos de N; y b. (20%) Qué imprimiría (déjelo explícito) si la entrada fuera la siguiente:

8
2 3 5 6 8 9

a: $O(N)$

i=1	e=2	[2]	I=1	e=2
i=2	e=3	[2,3]		
i=3	e=4	4		
i=4	e=5	[2,3,5]		
i=5	e=6	[2,3,5,6]		
i=6	e=7	7		
i=7	e=8	[2,3,5,6,8]		
i=8	e=9	[2,3,5,6,8,9]		

OTR:
2
5
8

1. (40%) Diseñe un algoritmo en Python o Java, para resolver el siguiente problema lo más eficientemente posible: Dado un arreglo estático A (de tipo tuple si es en Python o [] si es en Java), con N números enteros diferentes, el cual está ordenado ascendentemente pero rotado en algún índice de pivote k, definir una función que reciba A y N e imprima el máximo valor en el arreglo.

Así por ejemplo si A = 6, 7, 1, 3 el índice pivote k sería el 2, y el valor a imprimir sería 9

El pivote se entiende como el índice del arreglo original en el cual se hizo el "corte" y los elementos a la derecha de este pasaron al inicio del arreglo. En el ejemplo anterior el arreglo original sería 1, 3, 6, 7, 9.

Pista: Considerando que la función recibe el arreglo (es decir que no tiene que leer los valores), la eficiencia de la función puede ser menor a $O(N)$.

bscst_modify(A, N)

while True:

i = N//2

E = A[i]

if E > A[0]:

lado izquierdo

if A[i+1] < E:
return E

else:

A = [i:]

i //= 2

else:

lado derecho

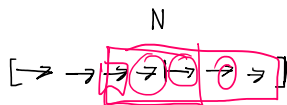
if A[i-1] > E:

return E:

else:

A = [:i]

i //= 2



```
def bs_improved(A):
    L = A
    while True:
        i = len(L)//2
        e = L[i]

        if e > L[0]:
            if L[i+1] < e:
                return e
            else:
                L = L[i:]

        else:
            if L[i-1] > e:
                return e
            else:
                L = L[:i]

print(bs_improved([10,11,14,18,1,2,4]))
```

3. Realice una prueba de escritorio al siguiente código y determine:

```
from collections import deque
N = int(input())
X = deque()
for i in range(N):
    X.append(int(input())) #Inserta al final de la lista
suma = 0
contador = 0
while len(X) > 0:
    elemento = X.popleft() #Hace un pop al inicio de la lista
    contador += 1
    if contador % N != 0:
```

[20, 30, 40, 50]

I, i ∈ [1, 4]

suma = 140

I, i ∈ [5, 8]

suma = 250

T, i ∈ [9, 12]

```

contador = 0
while len(X) > 0:
    elemento = X.popleft() #Hace un pop al inicio de la lista
    contador += 1
    if contador % N != 0:
        X.append(elemento)
    suma += elemento
print(suma)

```

A. (15%) Qué imprimiría si la entrada fuera:

4
20
30
40
50

B. (15%) Para un valor de N cualesquiera, cuál es su correspondiente eficiencia en notación Big Oh

$O(N^2)$

$I_1 \in [5, 8]$
 $suma = 250$

$I_2 \in [9, 12]$
 $suma = 390$

$I_3 \in [13, 16]$
 $suma = 510$

OTP: 510

1. (30%) Realice una prueba de escritorio al siguiente código y determine cuál es su eficiencia en notación big O, hágalo tanto línea por línea (de la 1 a la 10) como en total:

#	Línea	Eficiencia
0	from bisect import bisect_left	
1	N = int(input())	$O(1)$ ✓
2	X = list()	$O(1)$ ✓
3	X.append(0)	$O(1)$ ✓
4	X.append(1)	$O(1)$ ✓
5	for i in range(2, N):	$O(N)$ ✓
6	m = len(X)	$O(1)$ ✓
7	X.append(X[m-1] + X[m-2])	$O(1)$ ✓
8	for j in range(N):	$O(N)$ ✓
9	k = 2*j + 1	$O(1)$ ✓
10	print(bisect_left(X, k))	$\log_2(N)$ ✓
11	#bisect_left es búsqueda binaria y devuelve el índice más a la izquierda en que está el elemento k en X, o el índice donde podría agregarse en caso que no esté y que X siga ordenado	

Eficiencia total: $O(N \log N)$ ✓

2. (30%) Realice una prueba de escritorio al siguiente código:

```

from collections import deque
N = int(input())
X = deque()
for i in range(N):
    X.append(int(input())) #Inserta al final de la lista
s = 0
for i in range(N):
    s += X.popleft() #Hace un pop al inicio de la lista
    X.append(s)
while len(X) > 0:
    print(X.popleft())

```

$[0, 1, 12, 13, 14]$

$[1, 12, 13, 14, 10]$ 0

$[12, 13, 14, 10, 21]$ 1

$[13, 14, 10, 21, 33]$ 2

$[14, 10, 21, 33, 46]$ 3

$[10, 21, 33, 46, 60]$ 4

y determine que imprimiría si la entrada fuera:

5
10
11
12
13
14

10
21
33
46
60